

Reviewer Pack — Noncommutative L^p Norm Lower Bound for Disjointness Matrices...

Entry 4a2bf93b3b28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 13:29:47 UTC

Noncommutative L^p Norm Lower Bound for Disjointness Matrices

Entry ID: 4a2bf93b3b28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 13:29:47 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

The noncommutative L^p norm of the communication matrix M for DISJOINTNESS satisfies $\|M\|_p \geq \Omega(n)$ for $p = \log n$, and this norm lower bounds the randomized communication complexity of the function.

Rationale (proposer's reasoning):

Noncommutative L^p norms capture operator-theoretic structure of matrices, which may expose geometric barriers to efficient communication. For DISJOINTNESS, the matrix's high norm could reflect the need for $\Omega(n)$ bits to resolve disjointness via adaptive protocols.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6beb98a615cd0062

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For 30 seeds, compute $\|M\|_p / n$ for $p = \log n$. Support if min ratio ≥ 0.1 ; falsify if any seed's ratio < 0.1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    # Ensure the matrix is symmetric and has zeros on the diagonal
    for i in range(n):
        for j in range(i+1, n):
            M[j][i] = M[i][j]
            if random.choice([True, False]):
                M[i][j] = 0

    p = math.log2(n)

    # Compute the noncommutative L^p norm (using singular value decomposition)
    def svd(matrix):
        U, S, Vt = [], [], []
        m, n = len(matrix), len(matrix[0])
        for i in range(m):
            row = [matrix[i][j] for j in range(n)]
            U.append(row)

        for i in range(n):
            col = [matrix[j][i] for j in range(m)]
            Vt.append(col)

        S = sorted([sum(x**2 for x in row)**0.5 for row in U], reverse=True)
        return U, S, Vt

    U, S, Vt = svd(M)
    norm_p = sum(S[i]**p for i in range(min(len(S), 10)))*(1/p) # Use top 10 singular values for

    metric_value = norm_p / n
    instances_tested = 1
    conjecture_holds = metric_value >= 0.1
```

```

counterexample = "" if conjecture_holds else "Noncommutative L^p norm too small"

return {
    "metric_name": "noncommutative_Lp_norm",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = (sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))**0.5
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Noncommutative L^p norm too small\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE All trials used n=1")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

"conjecture_holds": True, "counterexample": ""
TRIAL: {"seed": 463, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.2858705913056837, "instances_tested": 1}
TRIAL: {"seed": 503, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.2918007642274359, "instances_tested": 1}
TRIAL: {"seed": 547, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.5421441477680837, "instances_tested": 1}
TRIAL: {"seed": 593, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.23203590952770686, "instances_tested": 1}
TRIAL: {"seed": 631, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.24900934832637536, "instances_tested": 1}
TRIAL: {"seed": 677, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.4523708904513098, "instances_tested": 1}
TRIAL: {"seed": 727, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.1730611083822355, "instances_tested": 1}
TRIAL: {"seed": 773, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.31022528349817463, "instances_tested": 1}
TRIAL: {"seed": 821, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.18516347328321206, "instances_tested": 1}
TRIAL: {"seed": 877, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.2670436577208097, "instances_tested": 1}
TRIAL: {"seed": 929, "metric_name": "noncommutative_Lp_norm", "metric_value": 0.3006046524354541, "instances_tested": 1}
RESULT: SUPPORTED mean=0.28556090944639073 std=0.09065176755208919 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n too small: all trials test instances_tested=1, which is insufficient to observe $\Omega(n)$ scaling.
Metric saturation: values ~ 0.285 are below 1, making $\Omega(n)$ impossible for $n \geq 2$. Selection
bias: single-instance testing misses adversarial cases.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test used $n=1$ instances, preventing observation of $\Omega(n)$ scaling. Metric values (~ 0.285) are below 1, making $\Omega(n)$ impossible for $n \geq 2$. | next: Test with $n \geq 1000$ and multiple instances to validate $\Omega(n)$ scaling

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	88989
2	preregistration	ollama_remote	qwen3:8b	0	0	22727
3	novelty	ollama_remote	qwen3:8b	0	0	24204
4	novelty	ollama_remote	qwen3:8b	0	0	19448
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11026
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9824
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8343
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9100
9	critic	ollama_remote	qwen3:8b	0	0	24228
10	judge	ollama_remote	qwen3:8b	0	0	26040

Totals: 0 input tokens, 0 output tokens, $\sim \$0.0000$ cost, 243929 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4a2bf93b3b28.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4a2bf93b3b28.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4a2bf93b3b28.tar.gz` (if generated)